10-K Agentic RAG - Technical Report

A local agent that routes between Text-to-SQL and PDF search, returning grounded answers

What I Built

A RAG system that answers natural-language questions over a financial dataset (6 10-K PDFs and a SQL database), drawing on SQL for quantitative data and the filings for narrative context, or both.

Architecture

A query goes to LlamaIndex's FunctionAgent, which calls one or both tools (query_financial, search_filings). Each call produces Evidence; these accumulate across turns into an AgentResponse with the answer and its sources. Evidence is a discriminated union, so answers stay auditable and inspectable.

Retrieval

Structured questions use LlamaIndex's NLSQLTableQueryEngine, which translates the question into a read-only SELECT and returns rows. I hand-wrote the schema context by inspecting the database, since column definitions alone aren't enough: it encodes what the model can't infer, like annual rows using period_type='FY' (there is no 'annual') and monetary columns being raw dollars. The executed query and rows are captured as evidence, so every number is reproducible.

Narrative questions use a vector store: pymupdf extracts each page into a LlamaIndex Document, SentenceSplitter chunks them (size 512), and each chunk is embedded with qwen3-embedding-8b. Retrieval is top-k with ticker, fiscal year, and page metadata.

Evaluation

No harness was provided, so measuring correctness over mixed exact/narrative answers was part of the work. I mirror how the system is graded so the dev score predicts the held-out score: the answer key tags each question with an evaluation type, and my harness implements one scorer per type — fuzzy numeric, exact entity match, and an LLM judge. The numeric scorer checks every number against the gold within a tolerance, so years and other incidental figures don't cause false matches. I also generate extra SQL questions with gold pulled straight from the database to stress-test beyond the ten.

The system scores 8/10, stable across runs. The failures are instructive: one asks for YouTube ad revenue, a figure that lives only in a PDF table (which extraction flattens), so the agent

declines rather than guessing; the other is a correct-but-incomplete answer the binary judge failed, motivating partial-credit scoring. The scoring logic is unit-tested with mocked model calls.

Trade-offs

- LLM text-to-SQL is flexible but non-deterministic; the schema context keeps it reliable and the agent retries failed queries.
- PDF extraction flattens tables, so all numeric questions go to SQL. Cost: figures only in PDF tables (e.g., YouTube ad revenue) are unreachable, the q_017 miss.
- Only the agent entry is async; retrieval is sync. Fine for one user, not concurrent load.
- VectorStoreIndex over a dedicated vector DB: simple for 6 PDFs, won't scale.
- The multi-step agent is thorough but slower than single-shot.

What I'd Improve

- **SQL validate-and-repair:** re-prompt when a query errors or returns no rows.
- **Table-aware PDF parsing:** extract tables (find_tables/LlamaParse) as markdown so their figures become retrievable.
- **Reranker:** add a cross-encoder step for PDF precision.
- **Latency:** dedupe chunks, async aquery, lower top-k, cap agent iterations.
- **Rubric scoring:** replace the binary judge with partial credit.

AI Usage

I designed and implemented most modules myself, using AI as a debugging partner when stuck for over an hour, mainly for run_eval, the text-to-SQL grounding, and parts of pdf.py.